

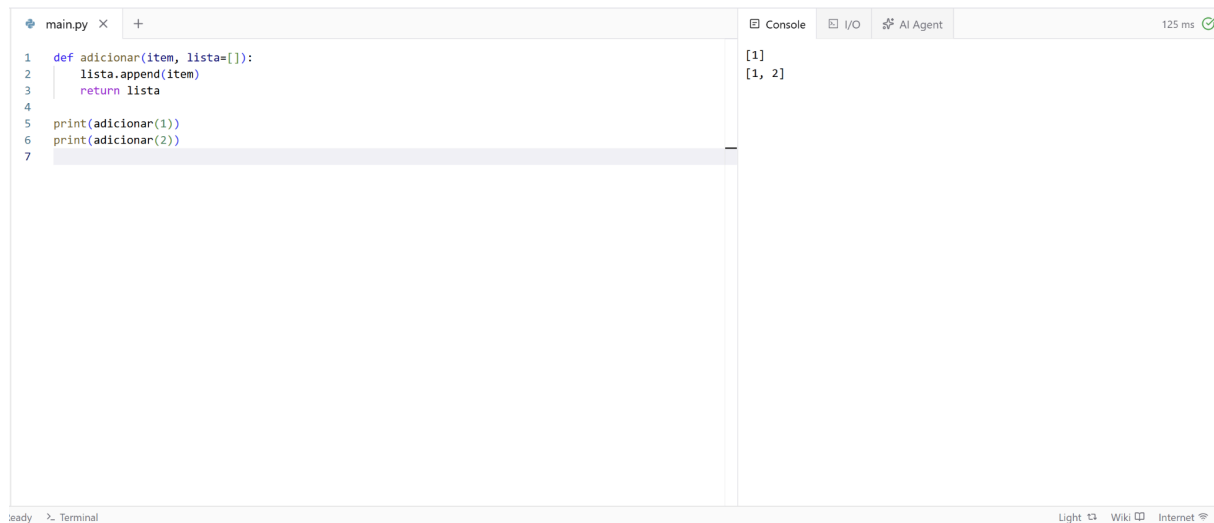
Atividade aula 008

1 - Python:

```
def adicionar(item, lista=[]):  
    lista.append(item)  
    return lista
```

```
print(adicionar(1))  
print(adicionar(2))
```

a-



```
main.py x +  
1 def adicionar(item, lista=[]):  
2     lista.append(item)  
3     return lista  
4  
5 print(adicionar(1))  
6 print(adicionar(2))  
7  
Console I/O AI Agent 125 ms  
[1]  
[1, 2]  
Ready Terminal Light Wiki Internet
```

b - **Valores padrão mutáveis.** Em Python, os argumentos padrão (como `lista=[]`) são avaliados e criados apenas *uma vez* em memória, durante a definição da função. Como a lista é mutável, a mesma lista sobrevive na memória e é reutilizada em todas as chamadas seguintes onde o parâmetro não for explicitamente passado.

2 - Java

```
static void zera(int[] v, int n) {  
    v[0] = 0;  
    n = 0;  
}  
int[] v = {5, 5}; int n = 5;  
zera(v, n);  
System.out.println(v[0] + " " + n);
```

a-

```

J Main.java x +
1 import java.util.*;
2
3 public class Main {
4
5     static void zera(int[] v, int n) {
6         v[0] = 0;
7         n = 0;
8     }
9
10    public static void main(String[] args) {
11        int[] v = {5, 5}; int n = 5;
12        zera(v, n);
13        System.out.println(v[0] + " " + n);
14    }
15 }

```

Console I/O AI Agent 991 ms

0 5

Light Wiki Internet

b- Passagem por valor em Java. Java passa tudo por valor. No caso do tipo primitivo `n`, é passada uma cópia do valor `5`; alterar a cópia dentro da função não afeta o `n` original de fora. No caso do array `v` (que é um objeto), passa-se a cópia da *referência* de memória. Como ambas as referências (a original e a da função) apontam para o mesmo array no Heap, alterar o conteúdo `v[0]` dentro da função altera o objeto visto por quem chamou.

3 - Python

```
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

a-

```

main.py x +
1 fs = [lambda: i for i in range(3)]
2 print([f() for f in fs])
3

```

Console I/O AI Agent 126 ms

[2, 2, 2]

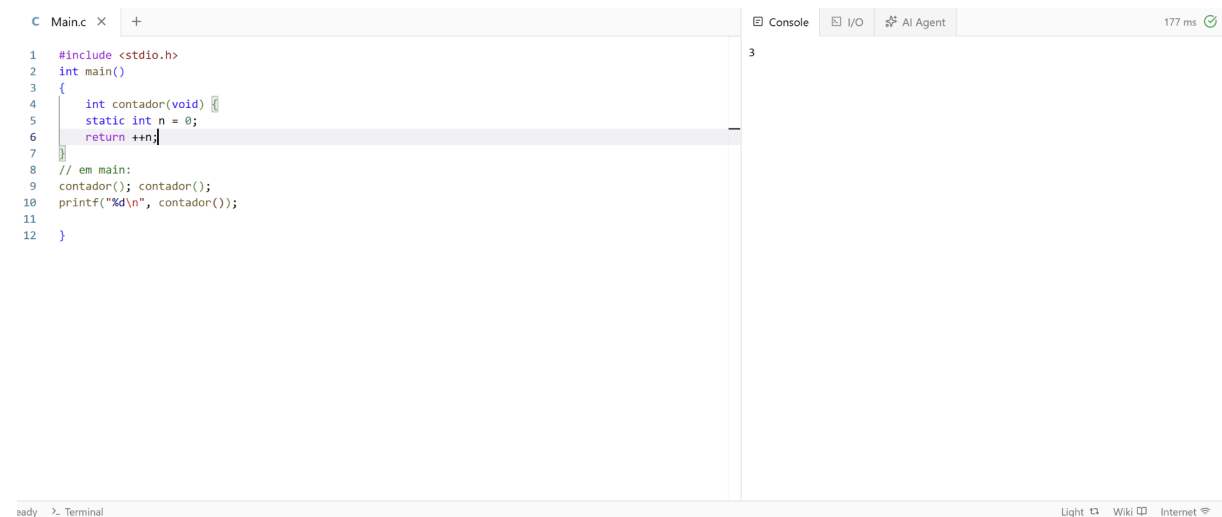
Light Wiki Internet

b- Fechamentos (closures) e captura por referência. Em Python, funções (incluindo lambdas) capturam as variáveis do escopo externo pela referência, e não copiando o valor imediato no momento da criação. Quando as funções são finalmente executadas no `print`, o laço `for` já terminou e a variável `i` parou em seu último valor (2). Todos os lambdas consultam a mesma variável e retornam 2.

4 - C

```
int contador(void) {  
    static int n = 0;  
    return ++n;  
}  
// em main:  
contador(); contador();  
printf("%d\n", contador());
```

a-



The screenshot shows a code editor with a file named 'Main.c'. The code defines a function 'contador' that takes no arguments and returns an integer. Inside the function, a static integer 'n' is declared and initialized to 0. The function returns '++n', which increments 'n' and returns the new value. In the 'main' function, 'contador' is called twice, and the result of the second call is printed using 'printf'. The console output shows the number '3'. The editor interface includes a 'Console' tab, 'I/O' and 'AI Agent' icons, and a '177 ms' execution time indicator.

```
1 #include <stdio.h>  
2 int main()  
3 {  
4     int contador(void) {  
5         static int n = 0;  
6         return ++n;  
7     }  
8     // em main:  
9     contador(); contador();  
10    printf("%d\n", contador());  
11  
12 }
```

b- **Ambiente de referenciamento local estático.** Variáveis declaradas como **static** em C não morrem na pilha de execução ao final da função. Elas vão para a área de dados estáticos da memória e preservam (lembram) seus valores entre chamadas consecutivas. Ao chamar a terceira vez, a variável que estava em 2 vira 3.

5 - Rust

```
fn dobra(v: Vec<i32>) -> Vec<i32> {  
    v.iter().map(|x| x * 2).collect()  
}  
let v = vec![1, 2, 3];  
let d = dobra(v);  
println!("{:?} {:?}", v, d);
```

a-

```

HelloWorld.rs x
1
2
3
4 fn main() {
5     let v = vec![1, 2, 3];
6     let d = dobra(v);
7     println!("{:?} {:?}", v, d);
8 }
9
10 fn dobra(v: [i32]) -> Vec<i32> {
11     v.iter().map(|x| x * 2).collect()
12 }

Console I/O AI Agent Exit 1
error[E0308]: mismatched types
--> HelloWorld.rs:6:19
6 |     let d = dobra(v);
  |               ^----- ^ expected `[i32]`, found `Vec<integer>`
  |               |
  |               arguments to this function are incorrect
  |
  = note: expected slice `[i32]`
           found struct `Vec<integer>`
note: function defined here
--> HelloWorld.rs:10:4
10 | fn dobra(v: [i32]) -> Vec<i32> {
   |     ^^^^^
error[E0277]: the size for values of type `[i32]` cannot be known at compilation time
--> HelloWorld.rs:6:19
6 |     let d = dobra(v);
  |               ^ doesn't have a size known at compile-time
  |
  = help: the trait `Sized` is not implemented for `[i32]`
  = note: all function arguments must have a statically known size

```

b- **Posse (Ownership) e movimentação (Move)**. Em Rust, passar uma variável por valor (como o vetor `v`) transfere a "posse" daquela memória para a função chamada. Quando a linha `println!` tenta usar o vetor original `v`, o compilador a impede, pois `v` foi "movido" para a função `dobra` e deixou de ser válido no escopo original.

6 - Python

```
total = 0
```

```
def adiciona(x):
    total = total + x
    return total
```

```
print(adiciona(5))
```

a-

```

main.py x +
1 total = 0
2
3 def adiciona(x):
4     total = total + x
5     return total
6
7 print(adiciona(5))
8

Console I/O AI Agent Exit 1
Traceback (most recent call last):
  File "/box/main.py", line 7, in <module>
    print(adiciona(5))
    ^^^^^^^^^
  File "/box/main.py", line 4, in adiciona
    total = total + x
    ^^^^^
UnboundLocalError: cannot access local variable 'total' where it is not associated with a value
Debug this error

```

b- **Regras de escopo locais/globais.** Em Python, ao fazer uma atribuição a um nome dentro de uma função (`total = ...`), o interpretador automaticamente assume que `total` é uma variável **local**. Como a instrução tenta ler o `total` local antes de a atribuição ser concluída (`total + x`), o programa quebra. Para funcionar, seria necessário declarar `global total` no início do corpo da função.